

Tabela de resumo de Python

1. TIPOS E VARIÁVEIS

Conversão entre tipos e verificação de tipo.

```
1 int("10"), float("3.14"), str(123), bool(0)
2 type(var), isinstance(var, int)
3
```

2. STRINGS (imutáveis)

Fatiamento, métodos comuns e formatação.

```
1 s = "python"
2 s[0:3], s[1:], s[::2] # fatias
3 s.upper(), s.lower(), s.strip() # maiusculo/minusculo/remover espacos
4 s.split(", "), ", ".join(lista) # dividir e unir
5 s.replace("py", "Java") # substituir
6 s.find("t") # indice (ou -1 se nao achar)
7 s.count("p") # conta ocorrencias
8 s.isdigit(), s.isalpha() # verifica se e' numero/letra
9 s.startswith("p"), "x" in s # verificacoes
10 f"Idade: {idade}" # f-string (mais comum)
11
```

3. LISTAS (mutáveis)

Acesso, métodos e compreensão de listas.

```
1 l = [1, 2, 3, 4]
2 l[0], l[-1], l[1:3] # acesso e fatia
3 l.append(5), l.insert(0,0) # adicionar no fim/posicao
4 l.pop(), l.pop(1), l.remove(2) # remover (ultimo, posicao, valor)
5 l.index(3) # posicao do primeiro 3
6 l.sort(), l.reverse() # ordenar e inverter
7 l.extend([5,6]) # concatenar outra lista
8 l.clear(), l.copy() # esvaziar e copiar
9 [x**2 for x in range(10) if x%2==0] # compreensao com filtro
10
```

4. TUPLAS (imutáveis)

Criação e desempacotamento.

```
1 t = (1, 2, 3) # ou t = 1,2,3
2 a, b = (1, 2)
3 a, *resto = (1,2,3,4) # *resto pega o restante
4
```

5. DICIONÁRIOS (dict)

Acesso seguro, iteração, compreensão.

```
1 d = {"nome": "Ana", "idade": 30}
2 d["nome"], d.get("sexo", "F") # get evita erro se chave nao existe
3 d["peso"] = 65 # adicionar nova chave
4 d.pop("idade") # remove chave e retorna valor
5 d.popitem() # remove e retorna ultimo par
6 d.update({"sexo": "F"}) # atualiza com outro dict
7 for k, v in d.items(): print(k,v) # iterar
8 d.keys(), d.values() # listar chaves/valores
9 {x: x**2 for x in range(3)} # compreensao: {0:0, 1:1, 2:4}
10
```

6. CONJUNTOS (set)

Operações de conjunto.

```
1 s = {1, 2, 3, 3} # -> {1,2,3} (remove duplicatas)
2 s.add(4), s.discard(2), s.remove(2) # adicionar/remover
3 s.update({5,6}) # adicionar varios
4 s.intersection_update({1,2}) # mantem apenas a interseccao
5 # uniao(), interseccao(&), diferenca(-), xor(^)
6
```

7. CONDICIONAIS

Estrutura if/elif/else e operador ternário.

```
1 if x > 0:
2     print("Positivo")
3 elif x == 0:
4     print("Zero")
5 else:
6     print("Negativo")
7 resultado = "Par" if num % 2 == 0 else "Impar"
8
```

8. LAÇOS (LOOPS)

For, while, enumerate, break/continue.

```
1 for i in range(5): # 0 a 4
2     for i in range(2, 10, 2): # 2,4,6,8
3         for idx, val in enumerate(lista): # com indice
4             while condicao: # loop condicional
5                 ...
6             break, continue, else (no loop) # controle de fluxo
7
```

9. FUNÇÕES

Definição, argumentos, lambda.

```
1 def soma(a, b=1): # b e opcional
2     return a + b
3
4 def varios(*args, **kwargs): # *args = tupla, **kwargs = dict
5     print(args, kwargs)
6
7 f = lambda x: x*2 # funcao anonima
8
```

10. COMPREENSÕES E GERADORES

Lista, dict, set e geradores (economizam memória).

```
1 [x for x in range(5)] # lista
2 {x: x*2 for x in range(3)} # dicionario
3 {x for x in "abacaxi"} # conjunto (remove duplicatas)
4 (x for x in range(10)) # gerador (lazy) -> next(g)
5 # Compreensao aninhada:
6 [(x,y) for x in range(2) for y in range(2)]
7
```

11. ENTRADA / SAÍDA / ARQUIVOS

```
1 nome = input("Digite: ") # entrada do usuario (sempre str)
2 print(f"Ola {nome}", end="\n") # saida com f-string
3
4 # Leitura de arquivo
5 with open("arquivo.txt", "r", encoding="utf-8") as f:
6     conteudo = f.read() # le tudo
7     linha = f.readline() # le uma linha
8     linhas = f.readlines() # le todas as linhas
9
10 # Escrita (sobrescreve)
11 with open("out.txt", "w") as f:
12     f.write("texto")
13
14 # Append (adiciona ao final)
15 with open("out.txt", "a") as f:
16     f.write("mais texto")
17
```

12. EXCEPTIONS

Captura de erros.

```
1 try:
2     x = int(input("Numero: "))
3 except ValueError as e: # erro especifico
4     print(f"Erro: {e}")
5 except ZeroDivisionError:
6     print("Divisao por zero!")
7 else: # executa se nao houver erro
8     print("Sem erros")
9 finally: # sempre executa
10     print("Sempre executa")
11
```

13. OPERADORES E BUILT-INS

Operadores lógicos e funções nativas.

```
1 # Operadores
2 is, is not, and, or, not, in, not in
3 if (n := len(lista)) > 5: ... # walrus (atribui e compara)
4
5 # Funcoes built-in (nativas)
6 len(), sum(), max(), min(), sorted()
7 list(zip([1,2], ['a','b'])) # combina listas
8 all(lista), any(lista) # todos True / algum True
9 map(func, iteravel) # aplica funcao a cada elemento
10 filter(func, iteravel) # filtra elementos
11
```

NUMPY

Criação de arrays

Diferentes formas de criar arrays NumPy.

```
1 import numpy as np
2 np.array([1, 2, 3])           # array 1D
3 np.array([[1,2], [3,4]])     # array 2D (matriz)
4 np.zeros((2,3))              # tudo 0
5 np.ones((3,2))               # tudo 1
6 np.eye(3)                    # matriz identidade 3x3
7 np.arange(0, 10, 2)           # sequencia [0,2,4,6,8]
8 np.linspace(0, 1, 5)          # 5 pontos entre 0 e 1
9 np.random.randn(3,4)         # normal (media 0, var 1)
10 np.random.randint(0, 10, size=5) # inteiros aleatorios
11
```

Atributos e forma

Propriedades do array.

```
1 arr.shape                    # (linhas, colunas)
2 arr.ndim                    # numero de dimensoes
3 arr.size                     # total de elementos
4 arr.dtype                   # tipo (int64, float64, etc)
5 arr.reshape(2,3)            # redimensiona (total igual)
6 arr.T                       # transposta
7
```

Indexação e fatia

Acesso a elementos e subconjuntos.

```
1 arr[0, 1]                   # linha 0, coluna 1
2 arr[:, 1]                   # todas linhas, coluna 1
3 arr[1, :]                   # linha 1, todas colunas
4 arr[1::2, :]                # linhas impares
5 arr[arr > 0.5]              # filtro booleano (retorna 1D)
6 arr[(arr > 0) & (arr < 1)]  # condicoes combinadas (use &, |, ~)
7 # Indexacao com listas: arr[[0,2], [1,3]]
8
```

Operações e manipulação

Operações matemáticas e manipulação de arrays.

```
1 arr + 1, arr * 2, arr ** 2   # operacoes elemento a elemento
2 np.sqrt(arr), np.exp(arr), np.log(arr)
3
4 # Estatisticas
5 arr.sum(), arr.mean(), arr.std()
6 arr.sum(axis=0)              # soma cada coluna (reduz dimensao)
7 arr.mean(axis=1)             # media cada linha
8
9 A @ B                        # multiplicacao de matrizes
10
11 # Broadcasting: arr 2x3 + vetor 1x3 -> vetor replicado para cada linha
12 # arr 2x3 + escalar -> escalar aplicado a todos
13
14 # Manipulacao
15 np.concatenate([arr1, arr2], axis=0) # empilha vertical
16 np.vstack([a,b]), np.hstack([a,b])  # vertical/horizontal
17 np.where(cond, x, y)               # condicional elemento a elemento
18 np.where(arr > 0, arr, 0)          # exemplo: zera valores negativos
19 np.unique(arr)                     # valores unicos
20
21 # CUIDADO: view vs copy
22 view = arr[:]                     # visao (altera o original!)
23 copia = arr.copy()                # copia real (segura)
24
```

PANDAS

Estruturas e I/O

Series (1D), DataFrame (2D), leitura/escrita.

```
1 import pandas as pd
2 s = pd.Series([1,2,3], index=['a','b','c'])
3 df = pd.DataFrame({
4     'nome': ['Ana','Bob'],
5     'idade': [25, 30]
6 })
7
8 # Leitura / escrita
9 df = pd.read_csv('arquivo.csv', sep=',', encoding='utf-8')
10 df = pd.read_excel('planilha.xlsx', sheet_name='Sheet1')
11 df.to_csv('saida.csv', index=False) # index=False remove a coluna de indice
12 df.to_excel('saida.xlsx', index=False)
13
```

Inspeção e seleção

Visualizar dados e selecionar colunas/linhas.

```
1 df.head(3), df.tail(2)      # primeiras/ultimas linhas
2 df.info()                   # tipos e nulos
3 df.describe()               # estatisticas (numericas)
4 df.shape, df.columns, df.index # informacoes estruturais
5
6 # Selecao
7 df['nome']                   # Serie (uma coluna)
8 df[['nome', 'idade']]       # DataFrame (varias colunas)
9 df.loc[0]                   # linha por indice (rotulo)
10 df.loc[0:2, 'nome']         # linhas 0-2, coluna 'nome'
11 df.iloc[0]                  # linha por posicao (0 = primeira)
12 df.iloc[0:3, 0:2]           # linhas 0-2, colunas 0-1
13
14 # Filtros condicionais
15 df[df['idade'] > 20]         # condicional
16 df.query("idade > 20")      # filtro com string (alternativa)
17 df['idade'].isin([20, 30])  # pertence a lista
18 df[df['nome'].str.startswith('A')] # filtro com string metodo
19
```

Tratamento de nulos e duplicados

Valores ausentes (NaN) e registros duplicados.

```
1 # Nulos
2 df.isnull().sum()           # conta nulos por coluna
3 df.dropna(axis=0)           # remove linhas com qualquer NaN
4 df.dropna(subset=['idade']) # remove se NaN na coluna 'idade'
5 df.fillna(0)                # preenche todos com 0
6 df['col'].fillna(df['col'].mean()) # preenche com a media da coluna
7 df.fillna(method='ffill')   # preenche com valor anterior
8
9 # Duplicados
10 df.duplicated()              # True nas linhas duplicadas
11 df.drop_duplicates(subset=['nome'], keep='first')
12
```

Criação, ordenação e strings

Manipulação de colunas e dados de texto.

```
1 df['nova_col'] = df['idade'] * 2 # nova coluna baseada em outra
2 df['categoria'] = df['idade'].apply(lambda x: 'Maior' if x>=18 else 'Menor')
3 df['sexo_cod'] = df['sexo'].map({'M':1, 'F':0}) # mapeamento
4 df.rename(columns={'nome':'Nome_Completo'}, inplace=True)
5 df.sort_values(by='idade', ascending=False)
6
7 # Operacoes com strings
8 df['nome'].str.upper()        # tudo maiusculo
9 df['nome'].str.contains('An') # contem 'An'?
10 df['nome'].str.len()          # tamanho da string
11 df['nome'].str.extract(r'(\d+)') # extrai numeros com regex
12
```

Groupby, Merge, Pivot, Datetime

Agrupamento, junções, tabelas dinâmicas, datas.

```
1 # Groupby (agrupamento)
2 df.groupby('categoria')['idade'].mean() # media por categoria
3 df.groupby('categoria')[['idade','peso']].agg(['mean', 'sum'])
4 df.groupby(['categoria', 'sexo']).size() # contagem por dois niveis
5 df.groupby('categoria').agg(
6     media_idade=('idade', 'mean'),
7     max_peso=('peso', 'max')
8 ) # agregacao nomeada
9 df.groupby('categoria')['idade'].transform('mean') # broadcasting
10
11 # Merge (juncao) - tipos: inner, left, right, outer, cross
12 pd.merge(df1, df2, on='chave', how='inner')
13 pd.merge(df1, df2, left_on='col1', right_on='col2')
14
15 # Concat (empilhamento)
16 pd.concat([df1, df2], axis=0) # empilha linhas (vertical)
17 pd.concat([df1, df3], axis=1) # empilha colunas (horizontal)
18
19 # Pivot Table (tabela dinamica) - mais flexivel que pivot()
20 pd.pivot_table(df, values='vendas', index='cidade', columns='ano',
21     aggfunc='sum', fill_value=0)
22 df.pivot(index='cidade', columns='ano', values='vendas') # alternativa
23
24 # Datetime (datas)
25 df['data'] = pd.to_datetime(df['data']) # converte para datetime
26 df['ano'] = df['data'].dt.year
27 df['mes'] = df['data'].dt.month
28 df['dia_semana'] = df['data'].dt.day_name()
29 df['semana_ano'] = df['data'].dt.isocalendar().week
30
```